

Documentation

Fetch_data.py:

Definition “get_data”:

The function takes a stock symbol and a time interval (e.g. 15 minutes) as inputs, downloads historical price data using the yfinance API, and removes missing values. It then computes log returns from the closing prices, which represent relative price changes over time. The function returns the log returns for modeling and the original stock data containing OHLC (open, high, low, close) prices and volume.

Prepare_inputs.py:

Definition “regression_data_preparation”:

The function constructs regression-based features from a rolling window of log returns. First, it computes the average volatility across all rolling windows to stabilize the signal-to-noise ratio calculation. Then, for each window, it performs a weighted linear regression over the past returns to estimate the trend (slope). The signal-to-noise ratio (SNR) is calculated as the absolute slope divided by the average volatility. Windows with weak or unstable signals (SNR below the minimum threshold or zero regression denominator) are discarded. For valid windows, the function returns a feature vector consisting of the trend slope, normalized trend, and SNR, along with the indices of the removed samples.

Definition "returns_data_preparation”:

The function prepares sequence-based inputs and labels from log returns. Using a sliding window, it creates input sequences of past returns and assigns the subsequent return as the prediction target. Samples that were previously removed during regression filtering are skipped. The function returns arrays of input return sequences and corresponding labels.

Definition “technical_data_preparation”:

The function generates technical indicator features from historical price data using a rolling window. For each valid window, it computes common indicators such as simple moving averages (SMA), exponential moving average (EMA), and relative strength index (RSI). Samples corresponding to removed indices are excluded to maintain alignment with other feature sets. The function returns the extracted technical indicator features.

Features.py:

Definition “load_data”:

The function downloads historical market data, computes log returns, and prepares model inputs by combining multiple feature types. It extracts regression-based features, technical indicators, and raw return sequences, while removing samples with weak or noisy signals based on a minimum signal-to-noise ratio (SNR). The function returns the final input feature matrix x and the corresponding target values y .

Definition “label_scale_data”:

The function standardizes the target values (labels) using a “RobustScaler”, reshaping them if necessary. It returns the scaled labels and the fitted scaler for later inverse transformation.

Definition “input_scale_data”:

The function scales different types of input features separately: regression-based features with “StandardScaler”, technical indicators with “MinMaxScaler”, and return sequences with “RobustScaler”. It then concatenates all scaled features into a single input array and returns it along with the list of fitted scalers.

Definition “prepare_dataset”:

The function orchestrates the full data preparation pipeline for model training. It loads and constructs features and targets, splits the data into training and test sets, and applies feature scaling. Both input features and target values are standardized to improve model training stability. The function returns scaled training and test datasets along with the fitted scaler.

Result.py:

Definition “plot_results”:

The function visualizes model predictions against the true values using a line plot. True values are shown in blue, predicted values in dashed red, with customizable title and axis labels. It helps quickly assess the model’s performance visually.

Definition “calculate_metrics”:

The function computes standard regression evaluation metrics between true and predicted values. It returns the root mean squared error (RMSE) and R-squared (R^2) score to quantify prediction accuracy and goodness of fit.

Model.py:

Purpose:

This is the main script of the project. It contains the definition, training, and evaluation of the predictive model. All the steps (loading data, scaling, creating tensors, training the network and calculating metrics) are executed here.

Model Architecture (PredictionModel):

1. **LSTM Layers:**

Three stacked LSTM layers process sequential input data with shape (batch_size, sequence_length, features). Each layer has a hidden size of 64, allowing the network to capture temporal dependencies across the input window. Dropout of 0.2 is applied between layers to reduce overfitting. Each LSTM layer outputs hidden states for every time step, which are then passed to the next layer.

2. **Layer Normalization:**

Layer normalization is applied to the LSTM output across the hidden dimension. This scales and shifts the hidden activations to have consistent mean and stabilize training.

3. **Fully Connected Head:**

The fully connected head transforms the last hidden state of the LSTM into the final prediction. The first linear layer reduces the dimension to 64 and applies ReLU activation for non-linearity. Dropout of 0.3 is applied to prevent overfitting. The final linear layer maps the 64-dimensional representation to a single output, representing the predicted next return.

Forward Pass:

The LSTM receives the input sequence and computes hidden states for each time step. Only the last hidden state (out[:, -1, :]) is extracted and normalized. The normalized output is passed through the fully connected head to produce the scalar prediction. This ensures the model summarizes the entire input window before output.

Training Procedure:

The model is trained using Mean Squared Error (MSE) loss, which measures the average squared difference between predicted and true returns. The Adam optimizer updates the model parameters with a learning rate of 0.1. Training runs for 400 epochs. In each epoch, the model performs a forward pass to compute predictions, calculates the loss, backpropagates gradients using loss.backward(), and updates the parameters with optimizer.step(). The optimizer.zero_grad() call clears previous gradients before the next update to prevent accumulation.

Improvements/to do list:

- Understand LSTM more, to enhance the network
- Understand the training process more (LR, Optimizers...), to enhance the training process
- Improve data, more and better-quality data
- Add more inputs to training
- Get a different data API, that offers more data points

AI Disclosure: Parts of this documentation and code were generated with AI assistance. All content was reviewed and verified by the developer.